Rocket Flight Simulator

Development of a One-Dimensional Rocket Flight Simulation in Python

Technical Report Version 1.0

Prepared by

Demetrius Prophette

Department of Aerospace Engineering

California State University, Long Beach

July 2026

Table of Contents

Introduction

This project describes the development of a one-dimensional rocket flight simulator capable of simulating the vertical ascent of a rocket, with graphs and data based on the rocket's provided parameters. The simulator accepts user-defined rocket parameters, measures the vehicle's flight profile, and visualizes it using graphs and information about the altitude, velocity, and acceleration during the duration of the flight. This project was constructed to strengthen and better my understanding of flight dynamics, numerical simulation, as well as software design. It also provides me with a strong foundation for more advanced aerospace projects. The simulator gives me the ability to develop skills for my future projects involving principles such as propulsion modeling, guidance and control, and navigation, as well as higher-fidelity flight dynamics. It uses physics and engineering concepts to model the vertical flight of a rocket using classical mechanics and established engineering principles.

Objectives

The primary objectives of this project are listed below:

- Calculate a rocket's altitude, velocity, acceleration, and flight time based on user-defined parameters
- Model the one-dimensional vertical trajectory of a rocket during ascent, coast, and descent
- Visualize the altitude, velocity, and acceleration as functions of time
- Show the effects of thrust, gravitational forces, and aerodynamic drag
- Demonstrate the application of Newton’s laws and engineering principles in rocket flight simulation
- Establish a strong foundation for future projects that connect to propulsion modeling and multi-degree-of-freedom trajectory simulation
- Create a modular Python codebase that can be expanded with higher-fidelity atmospheric models, propulsion, and flight dynamics
- Use classical mechanics and numerical methods to analyze the flight of a rocket

Assumptions and Scope

Assumptions

The first version of my rocket flight simulator is meant to model the movement of a single one-dimensional rocket along the vertical axis. The simulation parameters include the thrust, aerodynamic drag, and gravitational force to calculate the rocket's velocity, acceleration, and its altitude through its powered ascent, coast, descent, and landing. Multiple assumptions are used to make the model understandable and simple.

- The rocket only moves in the vertical direction
- The rocket's mass remains constant throughout the flight
- Upward motion and forces are defined as positive
- The drag coefficient remains constant
- Gravitational acceleration remains constant at 9.81m/s^2
- Air density stays constant at sea level
- Thrust remains constant during the specified burn time
- Thrust becomes zero right after burnout
- Wind and horizontal motion are not yet modeled
- Rotational motion is not modeled
- The model prevents the rocket from moving below ground before launch

- The simulation ends when the rocket returns to ground level
- A maximum simulation time prevents the model from infinitely looping during failed launches
- A parachute or recovery system is not yet included

Scope

With these assumptions, the simulator operates adequately to study the basic relationships among thrust, drag, weight, acceleration, altitude, and velocity. The results produced from the model should be taken as estimates from a simplified engineering model rather than exact predictions of a real rocket flight.

Physics Model

The rocket flight model is based on Newtonian mechanics/physics. In the process of each simulation step, the forces acting on the rocket are calculated and combined to determine the net force of the rocket. Newton's Second Law is then used to calculate the rocket's acceleration, which is integrated to determine the velocity and altitude over the duration of the flight.

Newton's Second Law

Used to calculate the rocket's acceleration

$$a = F_{net} / m$$

- a = acceleration(m/s^2)
- F_{net} = net force (N)
- M = rocket mass (kg)

Net Force

The net force acting on the rocket is calculated from the combined effects of the thrust, gravity, and aerodynamic drag

$$F_{net} = F_{thrust} + F_{weight} + F_{drag}$$

Weight

Weight acts downward throughout the flight and is calculated using the standard gravitational force equation

$$F_{weight} = -mg$$

- m = rocket
- g = 9.81 (m/s^2)

Note: The negative represents downward force

Aerodynamic Drag

Drag opposes the direction of motion and depends on three things: velocity, frontal area, and drag coefficient.

$$F_{drag} = 1/2\rho C_D A V^2$$

- ρ = air density
- C_D = drag coefficient
- A = frontal area
- V = velocity

Note: The simulator reverses the direction of the drag force on whether the rocket is ascending or descending.

Frontal Area

The frontal area is calculated assuming the rocket has a circular cross-section.

$$A = \pi(d/2)^2$$

- d = diameter

Thrust Model

Version 1 assumes constant thrust during motor burn.

During Burn:

$F_{thrust} = T$

After Burnout:

$F_{thrust} = 0$

Numerical Method

The simulation uses a fixed time-step numerical integration method to calculate the rocket's motion throughout the flight. During each simulation step, the model calculates thrust, weight, drag, net force, and acceleration. The calculated acceleration is used to update the rocket's velocity, and the updated velocity is used to calculate the new altitude. This continues to repeat until the rocket returns to ground level or the max time has been reached.

Velocity Update

The change in velocity over one time step is calculated from acceleration:

$$V_{n+1} = V_n + a_n \Delta t$$

- V_n = velocity at the current time step
- V_{n+1} = velocity at the next time step
- a_n = current acceleration
- Δt = fixed simulation time step

Altitude Update

After velocity is updated, altitude is calculated using the new velocity:

$$h_{n+1} = h_n + V_{n+1} \Delta t$$

- h_n = altitude at the current time step
- h_{n+1} = altitude at the next time step
- V_{n+1} = updated velocity
- Δt = fixed simulation time step

Time Update

Simulation time advances by one fixed time step:

$$t_{n+1} = t_n + \Delta t$$

- t_n = current time step
- t_{n+1} = next time step
- Δt = fixed simulation time step

Integration Method

Velocity is updated before altitude, so the simulator uses a semi-implicit Euler integration method. The simulator is simple to install and appropriate for the first version of the rocket flight simulator. Even though it is simple to implement, its accuracy depends on the selected time-step size. The smaller the time step, the fewer errors arise, but it increases the number of calculations required. Version one of the simulation currently runs on a fixed time step of 0.01 seconds.

The sequence during every iteration goes as follows:

1. Calculate thrust.
2. Calculate weight.

3. Calculate aerodynamic drag.
4. Calculate net force.
5. Calculate acceleration.
6. Update velocity.
7. Update altitude.
8. Advance simulation time.
9. Record the flight data
10. Check for launch, landing, or time-out conditions

Software Design and Implementation

The rocket flight simulator was created using a modular-based software structure in Python. Each file is responsible for running a specific part of the simulation, so the code stays organized, reusable, and expandable. Breaking up the project into independent modules makes future improvements, testing, and maintenance vastly easier.

Modules

rocket.py

Defines the Rocket class and stores the physical properties of the rocket, including length, diameter, mass, burn time, thrust, drag coefficient, and propellant type. It also calculates the rocket's frontal area.

constants.py

Stores physical constants used throughout the simulation, including gravitational acceleration, sea-level air density, and the simulation time step.

forces.py

Contains the physics functions used to calculate thrust, weight, aerodynamic drag, net force, and acceleration.

simulation.py

Performs the numerical simulation by repeatedly calculating the forces acting on the rocket, updating its motion, storing the flight history, and determining launch, landing, and failed-launch conditions.

main.py

Serves as the entry point of the program. It creates the rocket object, runs the simulation, prints the flight summary, and generates plots of altitude, velocity, and acceleration.

environment.py

This module is reserved for future atmospheric modeling. It provides a location for implementing variable atmospheric properties, such as changing air density with altitude, in future versions of the simulator.

Program Flow Diagram

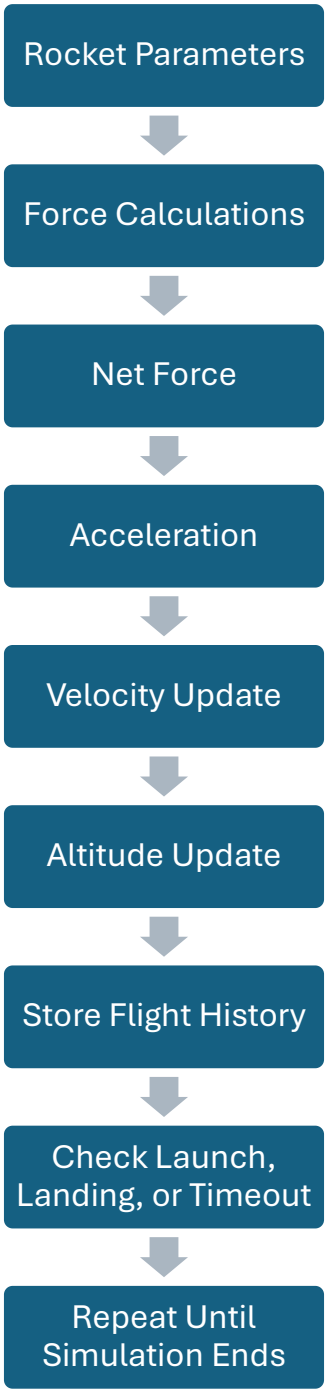

Figure 1 Program flow of the one-dimensional rocket flight simulator.

Results

The simulation was tested using a solid propellant rocket. The parameters below were used to evaluate the performance of Version 1 of the simulator.

Test Rocket Configuration

Parameter	**Value**
Rocket Name	Test Rocket
Mass	18 kg
Length	2.3 m
Diameter	0.102 m
Thrust	450 N
Burn Time	3.2 s
Drag Coefficient	0.65
Propellant Type	Solid

Flight Summary

```
Rocket Flight Summary

----------------------

Rocket Name: Test Rocket

Flight Time: 14.40 s

Maximum Altitude: 194.34 m

Maximum Velocity: 48.31 m/s
```

```
Maximum Acceleration: 15.19 m/s²
Impact Velocity: -60.71 m/s
```

Graphs

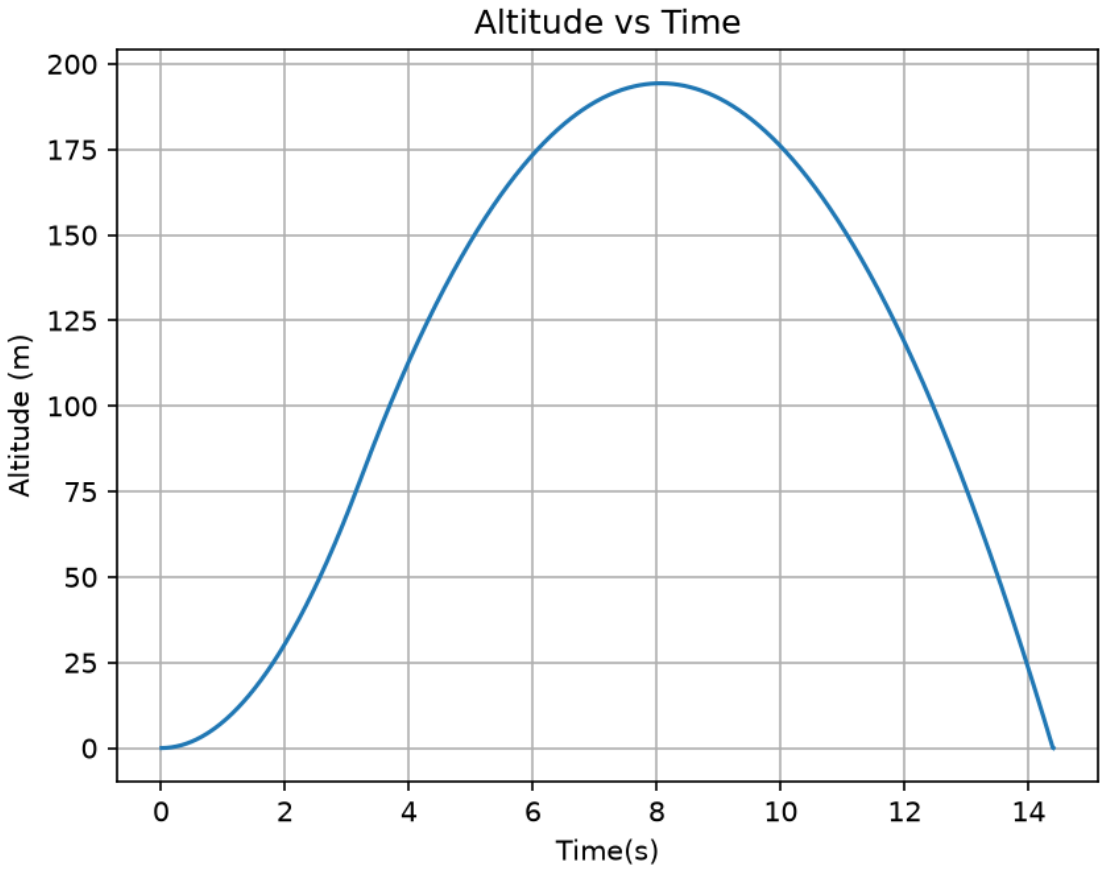

Figure 2 Altitude as a function of time.

The graph shows the altitude of the rocket accelerating during its powered ascent and even continuing to climb during its coasting phase, reaching a maximum altitude of approximately 194 meters before returning to ground level in its descent.

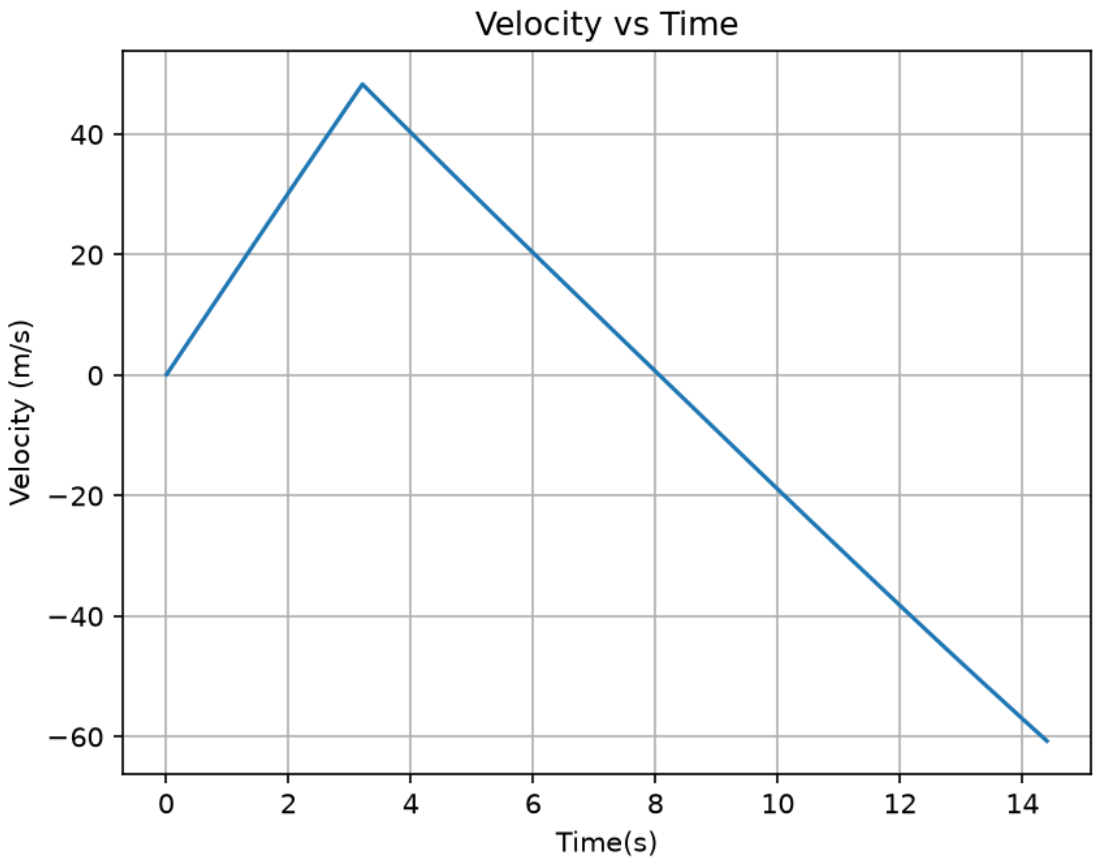

Figure 3 Velocity as a function of time.

During the duration of the flight, velocity climbs during the powered flight but starts decreasing during its coasting phase at around 3.5 seconds, right after the burn time of the rocket, 3.2 seconds. Comparing it to the altitude graph, it seems to reach 0 at apogee before becoming negative during its descent.

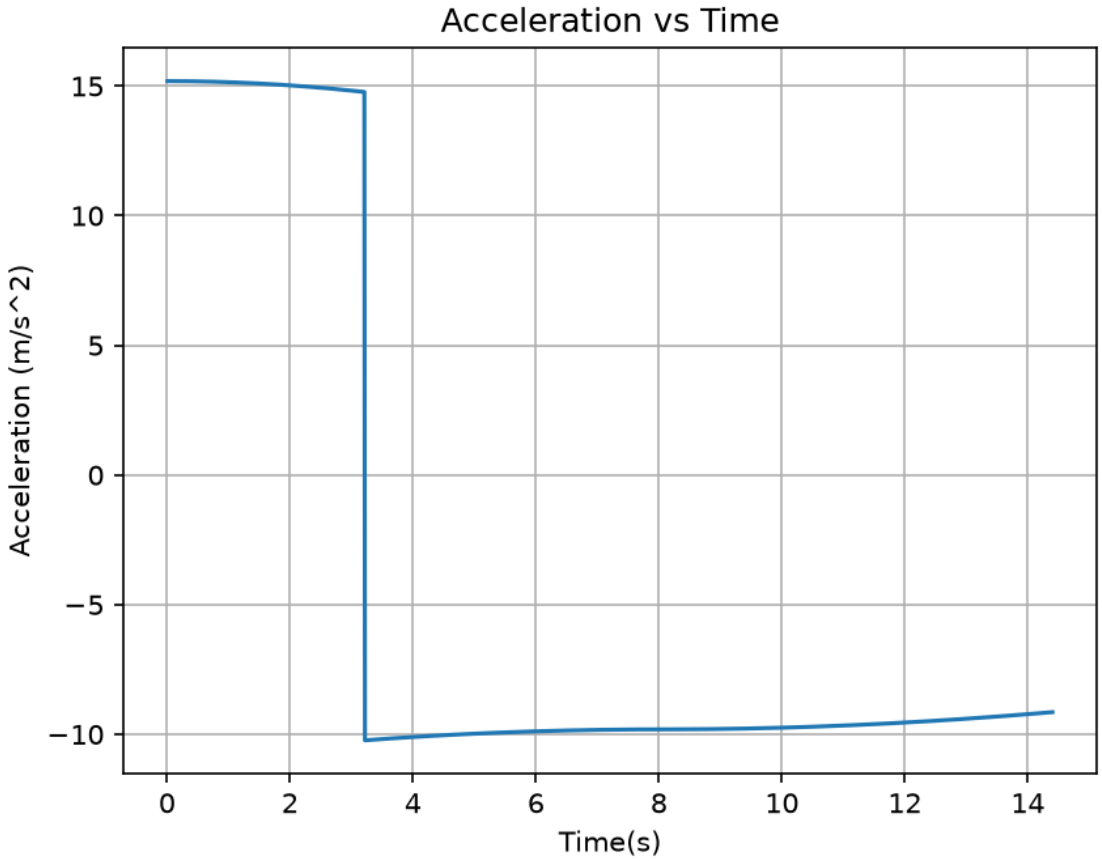

Figure 4 Acceleration as a function of time.

While the rocket is ascending, it experiences positive acceleration as the thrust from the motor surpasses the effects of both the gravitational and aerodynamic forces. Once the motor burns out, the acceleration decreases sharply once thrust is removed from the equation. This leaves only gravity and drag acting on the

rocket. During the coast phase and descent, the acceleration remains negative because gravity acts downward while aerodynamic drag opposes the rocket's motion, slightly reducing the magnitude of the downward acceleration.

Discussion

The results from the graphs are consistent with the assumptions and parameters given in the first version of the model. During the burn time of the motor, the rocket experiences positive acceleration because the thrust overcomes the effects of both drag and the weight of the rocket itself. This, in turn, causes both the altitude and velocity to increase during its powered ascent. After burnout, thrust will become zero and the rocket will continue to go upwards due to its residual velocity, while gravity and drag will slow it down. The velocity will eventually reach zero at apogee, and the rocket will begin to descend, making the velocity negative according to the simulations' upward positive sign convention.

During the rocket's descent, gravity is the dominant force while aerodynamic drag opposes the downward motion. The test rocket is heavy and has a small frontal area, making the drag manageable compared to its weight. This shows why the descent portion of velocity is nearly linear and why the downward acceleration remains close to gravitational acceleration. It takes 14.4 seconds for the rocket to reach ground level and has an impact velocity of -60.71 meters per second.

The sharp, sudden changes near burnout are due to the simplified thrust model in version 1. It goes from constant thrust to

zero thrust instantly during burn time, causing those sharp changes. A real rocket motor and a more advanced version of a rocket flight simulator would produce a thrust curve that depends on the burn and tapers near burnout. The results are adequate to show the fundamental behavior of a vertical rocket flight but should not be interpreted as an accurate prediction of an actual launch.

Limitations

Although the model provides a realistic approximation of one-dimensional rocket flight, multiple assumptions were made to keep the model computationally efficient and suitable for an introductory flight dynamics simulation. As a result of this, the simulation is not yet adequate to be considered a representation of a real rocket flight.

The limitations of the current version of the simulation:

- The simulation is only one-dimensional in the vertical direction and does not model horizontal or rotational movement.
- The rocket's mass continues to stay constant during the duration of the flight and does not decrease as propellant is consumed.
- Air density stays constant at sea-level conditions rather than varying with altitude.
- Gravitational acceleration is also assumed to stay constant during the simulation.
- The drag coefficient remains constant and does not change with velocity or Reynolds number.
- The thrust model assumes constant thrust during motor burn time and instantaneous transition to zero thrust at burnout.

- Wind effects and atmospheric disturbances are not included.
- Numerical integration is done using a fixed-time-step semi-implicit Euler method, which can cause approximation errors compared to higher-order integration methods.
- The simulator does not model parachute deployment or other recovery.

Future Work

Even though Version 1 successfully shows the fundamental principles of one-dimensional rocket flight simulation, there is an abundance of opportunities to improve both the physical accuracy and overall capability of the software. Future versions of the simulator will focus on expanding the fidelity of the flight model while keeping the modular software design established in this project.

Improvements for future versions:

- User-defined rocket parameters through an interactive input interface.
- Model the decrease in rocket mass as propellant is consumed during motor burn.
- Calculate atmospheric density as a function of altitude instead of assuming constant sea-level conditions.
- Replace the constant-thrust model with realistic motor thrust curves.
- Export flight data to external file formats such as CSV for additional analysis.

- Improve numerical accuracy using higher-order integration methods.
- Add wind and other atmospheric disturbances into the flight model
- Simulate parachute deployment and recovery during descent.
- Expand the simulator to two-dimensional and eventually six-degree-of-freedom flight dynamics.
- Integrate more advanced aerospace concepts such as propulsion modeling, guidance, navigation, and control (GNC).

The future improvements will bring this simulator from an educational one-dimensional flight model into a highly accurate aerospace engineering tool capable of supporting more advanced trajectory analysis and future rocketry projects.

Conclusion

This project successfully demonstrates the development of a one-dimensional rocket flight simulator that can model vertical motion of a rocket using classical mechanics and numerical simulation techniques. The model calculates the effects of various parameters such as gravity, aerodynamic drag, and thrust to measure the rocket's altitude, acceleration, and velocity during ascent, coast, descent, and landing. It shows the results in a flight summary of its key performance characteristics like flight time, maximum altitude, maximum velocity, maximum acceleration, and impact velocity, along with three graphs visualizing the altitude, velocity, and acceleration.

Beyond creating a functional simulation tool, the project strengthened my knowledge in flight dynamics, numerical integration, and engineering problem-solving skills. Organizing the simulator into modular Python components also set up a reusable foundation that can be expanded with more advanced physical models in future iterations.

Ultimately, Version 1 demonstrates that a one-dimensional model can effectively capture fundamental phases of rocket flight while laying a strong foundation for future improvements. The knowledge and skills gained from the project will serve as a strong stepping stone for more advanced simulations that involve higher fidelity and multiple degrees of freedom for the flight dynamics, along with atmospheric modeling, guidance and navigation, and variable mass propulsion.

References

Sutton, George P, and Oscar Biblarz. *Rocket Propulsion Elements*. John Wiley & Sons, 2001.

Anderson, John David Jr. *Introduction to Flight*. McGraw-Hill, 2016.

Hall, Nancy. “Guide to Aerodynamics.” NASA, 7 Dec. 2023.

Python Software Foundation. “The Python Tutorial — Python 3.7.4 Documentation.” 2019.

J. D. Hunter, "Matplotlib: A 2D Graphics Environment", Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.